

Universidad Distrital Francisco José de Caldas

Proyecto Curricular de Ingeniería en Sistemas

Curso de Arquitectura de Computadores

Práctica 3: Construcción de una ALU de 8 Bits con Circuitos Combinacionales

Docente:

Ing. Jonathan Roberto Torres Castillo, PhD.

Estudiantes:

Nombre del Estudiante 1

Nombre del Estudiante 2

03 de octubre de 2026

Índice

1. Introducción	2
2. Objetivos	2
3. Marco Teórico	2
3.1. Unidad Aritmético-Lógica (ALU)	2
3.2. Operadores Lógicos	3
3.3. Operadores Aritméticos	3
3.4. Multiplexor de 8 a 1 de 8 bits	3
4. Descripción del Desarrollo	3
5. Resultados	4
6. Análisis y Discusión	4
7. Conclusiones	4
8. Enlace a la Carpeta Compartida	5
9. Referencias	5

1. Introducción

La Unidad Aritmético-Lógica (ALU) constituye un componente fundamental en la arquitectura de los sistemas digitales[cite: 1], al encargarse de la ejecución de operaciones aritméticas y lógicas sobre datos binarios. Su diseño se basa en la integración de circuitos combinacionales que implementan funciones booleanas específicas, permitiendo la realización de operaciones como suma, resta y operaciones lógicas básicas (AND, OR, NOT, entre otras)[cite: 1].

Desde una perspectiva de diseño digital, la construcción de una ALU implica la descomposición de operaciones complejas en bloques funcionales elementales, los cuales pueden ser implementados mediante compuertas lógicas[cite: 1, 2]. Este enfoque permite comprender la relación entre las expresiones algebraicas booleanas, su representación mediante tablas de verdad y su materialización en circuitos físicos[cite: 2].

En esta práctica se aborda el diseño e implementación de una ALU de 8 bits utilizando exclusivamente compuertas lógicas y circuitos combinacionales, sin recurrir a bloques predefinidos[cite: 2]. Esto permite analizar de manera estructurada la construcción de sumadores, unidades lógicas y mecanismos de selección de operaciones, integrándolos en un sistema funcional que emula el comportamiento básico de una ALU[cite: 2].

2. Objetivos

- Comprender la estructura funcional de una ALU de 8 bits[cite: 2].
- Diseñar y construir circuitos combinacionales para operaciones aritmético-lógicas[cite: 2].
- Implementar un multiplexor de 8 a 1 de 8 bits sin componentes predefinidos[cite: 2].
- Integrar todos los módulos en un circuito completo de ALU[cite: 2].
- Evaluar el funcionamiento de la ALU mediante casos de prueba[cite: 2].

3. Marco Teórico

Los sistemas digitales se fundamentan en la manipulación de información binaria mediante circuitos combinacionales, los cuales implementan funciones booleanas a partir de compuertas lógicas[cite: 2].

3.1. Unidad Aritmético-Lógica (ALU)

La ALU es un bloque funcional encargado de ejecutar operaciones aritméticas y lógicas sobre datos binarios[cite: 2]. Su operación se basa en la recepción de operandos y una señal de control, la cual determina la operación que se debe realizar[cite: 2]. Es un circuito de tipo combinacional, lo que implica que sus salidas dependen únicamente de los valores actuales de sus entradas[cite: 2].

3.2. Operadores Lógicos

Los operadores lógicos permiten realizar transformaciones directas sobre variables binarias mediante reglas del álgebra de Boole, aplicándose bit a bit[cite: 3]:

- **AND:** $Y_i = A_i \cdot B_i$ [cite: 3]
- **OR:** $Y_i = A_i + B_i$ [cite: 3]
- **XOR:** $Y_i = A_i \overline{B_i} + \overline{A_i} B_i$ [cite: 3]
- **NOT:** $Y_i = \overline{A_i}$ [cite: 3]
- **XNOR:** $Y_i = A_i B_i + \overline{A_i} \overline{B_i}$ [cite: 3]

3.3. Operadores Aritméticos

- **Sumador completo (Full Adder):** Circuito combinacional que permite sumar tres bits de entrada (dos operandos y un acarreo de entrada C_{in}), produciendo un bit de suma (Sum) y un acarreo de salida (C_{out})[cite: 3].

$$Sum = A \oplus B \oplus C_{in} \quad (1)$$

$$C_{out} = AB + C_{in}(A \oplus B) \quad (2)$$

- **Resta en binario:** Se basa en el uso del complemento a dos, transformando la resta en una suma[cite: 3]:

$$A - B = A + (\overline{B} + 1) \quad (3)$$

- **Multiplicación binaria:** Se fundamenta en la generación de productos parciales mediante compuertas AND y su posterior suma con desplazamientos[cite: 4].

3.4. Multiplexor de 8 a 1 de 8 bits

Un multiplexor selecciona una entre 8 entradas utilizando 3 líneas de control (S_2, S_1, S_0)[cite: 6]. Para manejar una salida de 8 bits, se conectan 8 multiplexores en paralelo, compartiendo las mismas líneas de selección[cite: 6].

4. Descripción del Desarrollo

En esta sección se detallan los pasos seguidos para el diseño e implementación de cada módulo en **Logisim-Evolution**[cite: 6]:

1. **Componentes Básicos (Operaciones Lógicas):** Implementación mediante compuertas AND, OR, XOR, NOT y XNOR dispuestas en paralelo para buses de 8 bits[cite: 7].
2. **Sumador y Restador de 8 bits:** Encadenamiento en configuración *ripple-carry* de 8 sumadores completos de 1 bit[cite: 7]. Para la resta, se configuró la inversión de B con $C_{in} = 1$ [cite: 8].

3. **Multiplicador de 8 bits:** Diseñado mediante productos parciales y desplazamientos estructurales[cite: 8].
4. **Multiplexor y Integración:** Se acoplaron las salidas de las operaciones al MUX de 8 a 1 controlado por el Opcode ($S_2S_1S_0$)[cite: 8].

5. Resultados

A continuación se relacionan las operaciones de la ALU según el Opcode[cite: 9]:

Operación	Opcode ($S_2S_1S_0$)
AND	000
OR	001
XOR	010
NOT	011
XNOR	100
Resta	101
Suma	110
Multiplicación	111

Cuadro 1: Codificación del Opcode para la ALU[cite: 9]

Se verificó exitosamente el funcionamiento de la simulación. Por ejemplo, al realizar una multiplicación con las entradas $A = 00000011_2$ (3_{10}) y $B = 00000100_2$ (4_{10}) bajo el Opcode 111, se obtuvo la salida correcta $F = 00001100_2$ (12_{10})[cite: 9].

6. Análisis y Discusión

El diseño modular implementado permitió aislar posibles fallas en cada bloque antes de la integración final con el multiplexor[cite: 7, 8]. Una de las principales dificultades técnicas radicó en la correcta propagación del acarreo en el sumador/restador de 8 bits y en la gestión de los desplazamientos para la multiplicación[cite: 7, 8]. No obstante, el uso de circuitos combinatoriales puros garantizó respuestas síncronas estables sin retardos secuenciales indeseados.

7. Conclusiones

- La implementación jerárquica demostró que es posible construir sistemas complejos de procesamiento aritmético-lógico partiendo de compuertas básicas elementales[cite: 2, 7].
- El uso del complemento a dos optimizó los recursos de hardware al reutilizar la estructura del sumador de 8 bits para ejecutar operaciones de resta[cite: 3, 8].
- La arquitectura basada en un multiplexor de 8 a 1 permitió consolidar múltiples rutas de datos independientes en una única salida controlada mediante líneas de selección estables[cite: 6, 8].

8. Enlace a la Carpeta Compartida

`https://github.com/` (Reemplazar con el enlace real al archivo `.circ` en Logisim)[cite: 10, 11]

9. Referencias

Referencias

- [1] Mano, M. M., & Ciletti, M. D. (2017). *Digital Design* (6th ed.). Pearson[cite: 13].
- [2] Floyd, T. L. (2019). *Digital Fundamentals* (11th ed.). Pearson[cite: 13].
- [3] Roth, C. H., & Kinney, L. L. (2015). *Fundamentals of Logic Design* (7th ed.). Cengage Learning[cite: 13].
- [4] Logisim Evolution. (n.d.). *Open-source digital logic simulator*. Recuperado de `https://github.com/logisim-evolution/logisim-evolution`[cite: 13].